

Topic: Order Management System

1. Introduction

The Order Management System is used to store and manage customer orders and order details. It maintains information about customers, products, order dates, quantities, prices and order status. The system uses two main tables: ORDERS and ORDER_DETAILS. It also connects the orders with the existing CUSTOMER and PRODUCT tables using foreign keys.

2. Objectives

- To store customer order information.
- To store product details related to each order.
- To maintain order status.
- To calculate the subtotal of ordered products.
- To find the total quantity ordered for each product.
- To display customer-wise order information.
- To maintain data integrity using constraints.

3. Creating ORDERS Table

```
SQL> CREATE TABLE ORDERS (Order_ID VARCHAR2(10), Customer_ID VARCHAR2(10), Order_Date DATE, Total_Amount NUMBER(10,2), Order_Status VARCHAR2(20));  
Table created.
```

The ORDERS table stores the main information about each customer order.

4. Creating ORDER_DETAILS Table

```
SQL> CREATE TABLE ORDER_DETAILS (Order_Detail_ID VARCHAR2(10), Order_ID VARCHAR2(10), Product_ID VARCHAR2(10), Quantity NUMBER, Price NUMBER(10,2));  
Table created.
```

The ORDER_DETAILS table stores the products included in each order.

5. Adding Primary Key Constraints

```
SQL> ALTER TABLE ORDERS ADD CONSTRAINT PK_Orders PRIMARY KEY (Order_ID);  
Table altered.
```

Primary keys uniquely identify records in the tables.

6. Adding Foreign Key Constraints

```
SQL> ALTER TABLE ORDERS
  2  ADD CONSTRAINT FK_Orders_Customer
  3  FOREIGN KEY (CUSTOMER_ID)
  4  REFERENCES CUSTOMER(CUSTOMER_ID);
```

Table altered.

```
SQL> ALTER TABLE ORDER_DETAILS ADD CONSTRAINT FK_OD_Product FOREIGN KEY (Product_ID) REFERENCES PRODUCT(Product_ID);
```

Table altered.

These constraints maintain relationships between customers, orders and products.

7. Adding Data Validation Constraints

```
SQL> ALTER TABLE ORDERS MODIFY Order_Date NOT NULL;
```

Table altered.

```
SQL> ALTER TABLE ORDERS ADD CONSTRAINT CHK_Total CHECK (Total_Amount >= 0);
```

Table altered.

```
SQL> ALTER TABLE ORDERS ADD CONSTRAINT CHK_Status CHECK (Order_Status IN ('Pending', 'Confirmed', 'Ready', 'Delivered'));
```

Table altered.

```
SQL> ALTER TABLE ORDER_DETAILS ADD CONSTRAINT CHK_Qty CHECK (Quantity > 0);
```

Table altered.

```
SQL> ALTER TABLE ORDER_DETAILS ADD CONSTRAINT CHK_OD_Price CHECK (Price >= 0);
```

Table altered.

These constraints ensure valid order dates, non-negative amounts and prices, valid order statuses, and positive quantities.

8. Inserting Data into ORDERS

```
SQL> INSERT INTO ORDERS VALUES ('0001',101,TO_DATE('2026-09-01','YYYY-MM-DD'),3200,'Pending');
1 row created.

SQL> INSERT INTO ORDERS VALUES ('0002',102,TO_DATE('2026-09-03','YYYY-MM-DD'),1299,'Confirmed');
1 row created.

SQL> INSERT INTO ORDERS VALUES ('0003',103,TO_DATE('2026-09-05','YYYY-MM-DD'),1000,'Ready');
1 row created.

SQL> INSERT INTO ORDERS VALUES ('0004',104,TO_DATE('2026-12-23','YYYY-MM-DD'),1500,'Delivered');
1 row created.

SQL> INSERT INTO ORDERS VALUES ('0005',105,TO_DATE('2008-07-31','YYYY-MM-DD'),150,'Ready');
1 row created.
```

The query inserts order information using values entered during execution.

```
SQL> SELECT *FROM Orders;

ORDER_ID  CUSTOMER_ID ORDER_DAT  TOTAL_AMOUNT  ORDER_STATUS
-----
0001      101 01-SEP-26      3200 Pending
0002      102 03-SEP-26      1299 Confirmed
0003      103 05-SEP-26      1000 Ready
0004      104 23-DEC-26      1500 Delivered
0005      105 31-JUL-08       150 Ready
```

This query displays all records from the ORDERS table.

9. Inserting Data into ORDER_DETAILS

```
SQL> INSERT INTO ORDER_DETAILS VALUES ( '&ORDER_DETAIL_ID', '&ORDER_ID', '&PRODUCT_ID', &QUANTITY, &PRICE);
Enter value for order_detail_id: OD001
Enter value for order_id: 0001
Enter value for product_id: 11
Enter value for quantity: 100
Enter value for price: 3250
old 1: INSERT INTO ORDER_DETAILS VALUES ( '&ORDER_DETAIL_ID', '&ORDER_ID', '&PRODUCT_ID', &QUANTITY, &PRICE)
new 1: INSERT INTO ORDER_DETAILS VALUES ( 'OD001', '0001', '11', 100, 3250)

1 row created.

SQL> /
Enter value for order_detail_id: OD002
Enter value for order_id: 0002
Enter value for product_id: 12
Enter value for quantity: 60
Enter value for price: 1500
old 1: INSERT INTO ORDER_DETAILS VALUES ( '&ORDER_DETAIL_ID', '&ORDER_ID', '&PRODUCT_ID', &QUANTITY, &PRICE)
new 1: INSERT INTO ORDER_DETAILS VALUES ( 'OD002', '0002', '12', 60, 1500)

1 row created.

SQL> /
Enter value for order_detail_id: OD003
Enter value for order_id: 0003
Enter value for product_id: 13
Enter value for quantity: 150
Enter value for price: 300
old 1: INSERT INTO ORDER_DETAILS VALUES ( '&ORDER_DETAIL_ID', '&ORDER_ID', '&PRODUCT_ID', &QUANTITY, &PRICE)
new 1: INSERT INTO ORDER_DETAILS VALUES ( 'OD003', '0003', '13', 150, 300)

1 row created.
```

The query inserts product details into an order.

```
SQL> SELECT * FROM ORDER_DETAILS;
```

ORDER_DETA	ORDER_ID	PRODUCT_ID	QUANTITY	PRICE
OD001	0001	11	100	3250
OD002	0002	12	60	1500
OD003	0003	13	150	300
OD004	0004	14	50	2500
OD005	0005	15	30	399

This query displays all records from the ORDER_DETAILS table.

10. Customer Order Details Report

```
SQL> SELECT C.CUSTOMER_ID,C.CUSTOMER_NAME,O.ORDER_ID,O.ORDER_DATE,OD.PRODUCT_ID,OD.QUANTITY,OD.PRICE,OD.QUANTITY*OD.PRICE AS SUBTOTAL FROM CUSTOMER C JO
IN ORDERS O ON C.CUSTOMER_ID=O.CUSTOMER_ID JOIN ORDER_DETAILS OD ON O.ORDER_ID=OD.ORDER_ID ORDER BY C.CUSTOMER_ID,O.ORDER_DATE;

no rows selected
```

This query combines CUSTOMER, ORDERS and ORDER_DETAILS tables and displays customer name, order ID, order date, product ID, quantity, price and subtotal. The subtotal is calculated as Quantity × Price.

11. Total Quantity of Each Product

```
SQL> SELECT P.PRODUCT_NAME, SUM(O.QUANTITY) AS TOTAL_QUANTITY FROM PRODUCT P  
JOIN ORDER_DETAILS O ON P.PRODUCT_ID=O.PRODUCT_ID GROUP BY P.PRODUCT_NAME O  
RDER BY P.PRODUCT_NAME;
```

```
PRODUCT_NAME
```

```
-----  
-----
```

```
TOTAL_QUANTITY
```

```
-----
```

```
Jasmine  
150
```

```
Rose Bouquet  
30
```

```
White Lily  
60
```

```
Purple Orchid  
50
```

```
Red Rose  
100
```

This query calculates the total quantity ordered for each product using the SUM() function.

12. Customer Order Summary

```
SQL> SELECT C.CUSTOMER_NAME,O.ORDER_ID,O.ORDER_DATE,O.TOTAL_AMOUNT FROM CUSTOMER C JOIN ORDERS O ON C.CUSTOMER_ID=O.CUSTOMER_ID;
```

```
CUSTOMER_NAME
```

```
-----
```

```
ORDER_ID  ORDER_DATE  TOTAL_AMOUNT
```

```
-----
```

```
Ananya  
0001      01-SEP-26      3200
```

```
Priya  
0002      03-SEP-26      1299
```

```
Kaviya  
0003      05-SEP-26      1000
```

```
Karu  
0004      23-DEC-26      1500
```

```
Gobi  
0005      31-JUL-08       150
```

This query displays customer name, order ID, order date and total amount.